

MODULE 01 · DOMAIN 1 - AGENTIC ARCHITECTURE & ORCHESTRATION

The Agentic Loop

What an agent actually is, how the loop terminates, and the situations where you should not build one.

DOMAIN 1 of 5 EXAM WEIGHT 27% MODULE 1 of 20

An agent is not a model. An agent is a **loop** wrapped around a model.

The canonical loop

01

Gather context

Pull in what the model needs to decide the next step.

02

Take action

The model chooses a tool. Your code executes it.

03

Verify work

Check the result against something external.

04

Repeat

Feed the outcome back and go round again.

ON THE EXAM

Anthropic uses this four-step framing for the Claude Agent SDK. Learn it as four steps, because questions attack each one separately.

Three things make a system agentic

- ◆ **Tool use.** The model can act on the world, not just describe it.
- ◆ **Autonomy over the path.** The model chooses what to do next, and that choice depends on what it just learned.
- ◆ **A feedback loop.** Results come back into context and change the next decision.
- ◆ Remove any one of the three and you have something else, usually a workflow. That is often the right answer.

ON THE EXAM

Autonomy over the path is the discriminator. Tool use alone does not make an agent.

Underneath every pattern is one thing: a model with **retrieval, tools and memory** attached.

Workflow or agent?

WORKFLOW

Your code owns the path

- Models and tools follow predefined code paths
- Same steps, same order, every time
- Predictable, cheap, easy to test
- You can guarantee a step ran

AGENT

The model owns the path

- The model directs its own process and tools
- Next step depends on what it just found
- Flexible, slower, costlier, less repeatable
- You cannot guarantee a step ran

THE TIEBREAKER

Can you draw the flowchart in advance? If yes, it is a workflow. If the flowchart depends on what step two returns, it is an agent.

Three shapes, and what each is for

SHAPE	THE MODEL'S JOB	CHOOSE IT WHEN
Conversational	Answer, using context you supply	A person is in the loop every turn and no external action is needed
Workflow	Do one well-defined step of a path you designed	The steps are known, the order is fixed, and predictability matters
Agent	Decide the path and act on it	The path depends on what is discovered along the way

ON THE EXAM

Scenario questions often describe a workflow and offer an agent as the impressive-sounding wrong answer.

When not to build an agent

Anthropic's own guidance, and a reliable source of exam answers.

- ◆ **Start with the simplest thing that works.** For many applications a single call with retrieval and good examples is enough.
- ◆ **Agentic systems trade latency and cost for task performance.** That trade is not always worth making.
- ◆ **A fixed path does not need a decision-maker.** Adding one adds variance and nothing else.
- ◆ **Add complexity only when the simpler version has demonstrably failed.**

ON THE EXAM

"Should we use an agent here?" is a real exam question, and "no" is a real answer.

The loop does not stop when the model says it is finished. It stops when the model **stops asking for tools.**

One turn, in code

01

Send

Messages array plus tool definitions to the Messages API.

›

02

Read stop_reason

The field on the response says why generation stopped.

›

03

Execute

If it is tool_use, run the requested tools in your own code.

›

04

Append

Assistant turn verbatim, then a user turn of tool_result blocks.

›

05

Repeat

Call again. Exit when stop_reason is end_turn.

ON THE EXAM

`while stop_reason == "tool_use".` That is the entire loop condition.

The seven stop_reason values

VALUE	WHAT IT MEANS	WHAT YOUR LOOP DOES
<code>end_turn</code>	Finished naturally	Exit. Use the response.
<code>tool_use</code>	Wants tools executed	Run them, append results, call again.
<code>max_tokens</code>	Output budget exhausted	Raise the budget, or continue.
<code>stop_sequence</code>	A custom stop string matched	Read which one fired.
<code>pause_turn</code>	Server tool loop hit its cap	Append the response, call again.
<code>refusal</code>	Claude declined	Do not auto-retry. Log and surface.
<code>model_context_window_exceeded</code>	Request exceeds the window	Send less. A bigger number will not help.

ON THE EXAM

The three people forget: `pause_turn`, `refusal`, `model_context_window_exceeded`.

max_tokens versus model_context_window_exceeded

MAX_TOKENS

You set the ceiling too low

- Output ran past the limit you set
- Raising max_tokens can genuinely fix it
- Or continue generation from the cut
- The input was never the problem

MODEL_CONTEXT_WINDOW_EXCEEDED

The whole request is too big

- Input plus output exceed the window
- Raising a number does nothing
- Send less: trim, summarise, split
- Tool definitions count towards this

THE TIEBREAKER

One is a ceiling you chose. The other is a wall you cannot move. The fixes are opposites.

The truncated tool call

A specific failure worth recognising on sight.

- ◆ **stop_reason is max_tokens and the last block is a tool_use.** The tool's JSON input was cut off mid-structure.
- ◆ **Retrying with the same budget fails identically.** The model will generate the same tokens and hit the same wall.
- ◆ **Raise the budget, or reduce what the tool call has to carry.**
- ◆ The same shape appears in Domain 4 when a large extraction is truncated mid-JSON. Same cause, different symptom.

ON THE EXAM

"Retrying produces identical failures" in a question stem is a signal that the fix is structural, not a retry.

The messages array is not free-form

Three mutation rules the API enforces, and one it does not.

- ◆ **Append the assistant turn verbatim**, `tool_use` blocks included. Dropping it produces an invalid array and an API error.
- ◆ **The following user turn contains `tool_result` blocks only**. Each keyed by its `tool_use_id`.
- ◆ **No extra text alongside tool results**. It teaches Claude to expect user text after every tool call, and you get empty responses.
- ◆ **The Messages API is stateless**. You resend the whole conversation every turn. It remembers nothing.

ON THE EXAM

The last point drives half of Domain 5. If history is not in the request, it does not exist.

Five ways to write the loop wrong

- ◆ **Parsing the text for completion.** Looking for "done" or "finished" in the response instead of reading `stop_reason`.
- ◆ **Using an iteration cap as the stop condition.** A cap is a safety net, not a terminator.
- ◆ **Dropping the assistant turn** before appending tool results.
- ◆ **Trailing text after `tool_result` blocks.**
- ◆ **Treating `pause_turn` like `end_turn`,** which silently truncates every agent that uses server-side tools.

ON THE EXAM

The first one is the most common in exam distractors. It looks reasonable and it is a bug.

Terminator or guardrail?

THE TERMINATOR

No tool_use in the response

- stop_reason comes back as end_turn
- This is the loop's real exit condition
- Claude judged the task complete
- Everything else is protection

GUARDRAILS

Caps that abort the run

- max_turns caps iterations per invocation
- max_budget_usd caps spend per invocation
- Enforced by the SDK, not the model
- Hitting one means failure, not success

THE TIEBREAKER

If a run ends on a guardrail, nothing was completed. Your orchestration layer has to notice that difference.

Take away verification and an agent becomes a very expensive way to **generate plausible work.**

What verification looks like

- ◆ **Deterministic checks first.** A test suite, a linter, a JSON schema, a type checker. Cheap and unarguable.
- ◆ **Rules the agent cannot talk its way past.** A hook that blocks, a limit enforced inside the tool.
- ◆ **A second model pass** with fresh context and a narrow question. Useful, but it is judgement checking judgement.
- ◆ **Asking the agent whether it did a good job** is not verification. It is the same context grading itself.

ON THE EXAM

Prefer the cheapest check that cannot be argued with. Model-based review is the fallback, not the default.

Reading the question: which shape does it want?

WHAT THE STEM SAYS

WHAT IT IS TELLING YOU

"the same three steps for every request"

Workflow. A fixed path needs no decision-maker.

"depends on what it finds at each step"

Agent. The path cannot be written in advance.

"must be predictable and auditable"

Workflow, or an agent with hard guardrails.

"open-ended", "the engineer hasn't specified"

Agent. Nobody can draw this flowchart.

"guarantee", "regardless of prompting"

Neither. A control outside the model.

ON THE EXAM

Read the stem for these before the options. They eliminate two answers on sight.

No prompt makes an agent reliable. An agent plus a **control that runs whatever the agent does** can be.

Module 1 in five lines

- ◆ An agent is a loop your code owns. Gather context, take action, verify, repeat.
- ◆ Workflow when you can draw the flowchart in advance. Agent when you cannot.
- ◆ The loop runs while stop_reason is tool_use and exits on end_turn. Never parse text for completion.
- ◆ max_turns and max_budget_usd are guardrails. Hitting one is a failure, not a finish.
- ◆ Prompts persuade. Code enforces. When a question says guarantee, choose code.

ON THE EXAM

Five worked scenarios next. Commit to an answer before the walkthrough, every time.

A logistics company has built an agent that answers questions about shipments. It has four tools and works well in testing.

In production, roughly one call in twenty never returns. The loop keeps calling the API long after the agent has produced a complete answer, until the turn cap aborts it. Logs show the final responses contain no tool calls at all, only text. The loop implementation checks whether the response text contains the words "task complete" before exiting.

What is the root cause?

- ☐ A The turn cap is set too low for the workload
- ☐ B The loop determines completion from response text rather than from `stop_reason`
- ☐ C The agent needs a system prompt instructing it to say "task complete" when finished
- ☐ D Tool results are being appended without the preceding assistant turn

◆ PAUSE AND DECIDE

What is the root cause?

- A

 Turn cap too low
The cap is what stopped the runaway. It is the symptom's limiter, not the cause. Raising it makes the failure more expensive.
- B

 Completion read from text
The response has no tool calls, so stop_reason is end_turn and the loop should already have exited.
- C

 Prompt it to say "task complete"
Tempting, and it would reduce the failure rate. It makes correctness depend on phrasing, which is exactly the bug.
- D

 Missing assistant turn
That produces an API error immediately, not a loop that runs too long. The symptom does not match.

THE SIGNAL

"The final responses contain no tool calls" is the planted detail. No tool calls means end_turn, and end_turn means the loop had its exit and ignored it.

A team runs an automated review on every pull request. The review always does the same three things in the same order: check style against the house guide, scan for known security patterns, and confirm public functions are documented. Each part is independent of the others.

The current implementation gives a single agent all three checklists and lets it decide how to proceed. Results are inconsistent: sometimes it skips documentation entirely, and reviewers cannot tell whether a missing finding means clean code or a skipped step.

What is the most appropriate design change?

- A Add emphasis to the system prompt instructing the agent to always perform all three checks
- B Restructure as prompt chaining: three sequential steps, each a separate call with its own focused prompt
- C Give the agent three separate tools, one per check, and let it call them
- D Increase max_turns so the agent has room to complete all three checks

◆ PAUSE AND DECIDE

What is the most appropriate design change?

- A

 Emphasise in the prompt
Reduces skipping without eliminating it. The team needs to know a step ran, and a prompt cannot tell them that.
- B

 Prompt chaining
Fixed steps, fixed order, independent parts. Your code runs all three, so a missing finding provably means clean code.
- C

 Three tools, agent decides
Better than one blob, but the agent still owns the path, so it can still skip one. Same failure, smaller.
- D

 Raise max_turns
Nothing here suggests it ran out of turns. It chose not to do the work.

THE SIGNAL

Three fixed steps in a fixed order, independent of each other, is the textbook description of prompt chaining. There is nothing for an agent to decide.

An agent extracts contract terms and writes them into a records system through a single tool with a large structured input.

On long contracts, the run fails. The response comes back with `stop_reason` set to `max_tokens` and the last content block is a partial `tool_use` whose JSON input is cut off mid-object. The team's retry logic resends the identical request up to three times. Every attempt fails the same way.

What is the most effective fix?

- ☐ A Add exponential backoff between the three retries
- ☐ B Catch the parse error and have the agent summarise the contract in prose instead
- ☐ C Raise `max_tokens`, and split very long contracts across several tool calls
- ☐ D Reduce the number of tools available so more of the budget is left for output

◆ PAUSE AND DECIDE

What is the most effective fix?

- A

 Exponential backoff
Backoff helps with transient failures. This one is deterministic: the same input produces the same overflow every time.
- B

 Fall back to prose
Abandons the structured output the records system needs. It converts a hard failure into a silent one.
- C

 Raise the budget and split long inputs
Addresses the actual constraint, and splitting handles contracts that would overflow any budget.
- D

 Fewer tools
Tool definitions do consume input tokens, but max_tokens caps output. Right instinct, wrong limit.

THE SIGNAL

"Every attempt fails the same way" rules out every retry-shaped answer. Deterministic failures need a structural fix.

A support agent handles returns. Its tools include `get_customer`, `lookup_order`, `process_refund` and `escalate_to_human`.

A customer asks about returning an item. The agent calls `lookup_order` and the result shows the item was purchased forty-five days ago, against a thirty-day return policy stated in the system prompt.

A colleague asks how the loop decides whether to call `process_refund` or `escalate_to_human` at this point.

How is that decision made?

- | | |
|--|--|
| <p>A The SDK compares the tool result against the policy and routes to the correct tool</p> | <p>B The order details are added to the conversation and the model reasons about which action to take</p> |
| <p>C Your orchestration code inspects the purchase date and calls the appropriate tool directly</p> | <p>D <code>lookup_order</code> returns a routing hint that determines the next tool call</p> |

◆ PAUSE AND DECIDE

How is that decision made?

- A

 The SDK routes it
No SDK reads your policy. The SDK moves messages and runs tools; it has no view of business rules.
- B

 Result enters context; the model reasons
This is what the agentic loop is. Results become context, and the next decision is made from that context.
- C

 Your code decides
Describes a workflow, not an agent. Legitimate design, but it is not what the scenario built.
- D

 A routing hint in the result
The tool would then own the decision, which defeats the point of having a model in the loop.

THE SIGNAL

This is a definitional question wearing a scenario's clothes. Tool results are context. Context drives the next choice. That is the whole mechanism.

A media company wants to automate first-line triage of inbound press enquiries. Roughly ninety-five per cent fall into four categories that a keyword rule already classifies correctly today, and each category has a fixed template reply.

The remaining five per cent are genuinely unusual and currently go to a person. Volume is high, latency matters, and the finance team is asking why the AI budget has grown.

An engineer proposes replacing the whole system with an agent that has tools for classification, reply drafting and escalation.

What should you advise?

- ☐ A Build the agent. It handles both the common and unusual cases in one system, simplifying the architecture
- ☐ B Keep the rule for the ninety-five per cent and route only the unclassified remainder to a model
- ☐ C Build the agent but cap it with `max_turns` and `max_budget_usd` to control cost
- ☐ D Replace the keyword rule with a single classification call, keeping the templates and the human escalation

What should you advise?

- A

 Build the agent
Puts a slow, costly decision-maker in front of 95% of traffic that a free rule already handles correctly.
- B

 Keep the rule, escalate the remainder
Simplest thing that works. The model is spent only where the path is genuinely unknown.
- C

 Agent with caps
Caps limit the damage from a design that should not exist. They do not make a wrong architecture right.
- D

 Model classifier, keep templates
Adds cost and latency to replace something already correct. Fixes what is not broken.

THE SIGNAL

"A rule already classifies them correctly" is the planted detail. Nothing is broken about the ninety-five per cent, so nothing there needs a model.